

Rclone Backup to Box for Cluster

Markus G. S. Weiss

2025/05/05

Contents

1	Introduction	2
2	Prerequisites	2
3	Configure the Box remote with offline authorization	2
4	Create the Box folder hierarchy	4
5	Prepare the local environment	4
6	Reference scripts	4
6.1	A) backup.sh	5
6.2	B) crontab-backup	16
7	Install the cron job	16
8	Monitoring & Maintenance	17
9	Additional Notes	18
10	Summary	18

1 Introduction

This tutorial explains how to configure `rclone` on your cluster to back up `/groups/sterling/mfshome/$USER` to a Box directory named `cluster-backup`, with subfolders for `daily`, `archive`, and `logs`, and how to schedule it via `cron`. Users in the `sterling` group only need to run the commands in Sections 3–5 and 7. The scripts are maintained centrally under `/groups/sterling/hpc-setup`.

2 Prerequisites

- `rclone` (v1.38 or later) installed on the cluster
- A Box Enterprise SSO account
- Shell access to the cluster with `cron` available
- Web browser access (for OAuth authentication)

Tip

Before running any live syncs, you can test with `-dry-run` to see what would transfer or delete without affecting Box:

```
/groups/sterling/software-tools/rclone/rclone-v1.69.1-  
linux-amd64/rclone sync \  
/groups/sterling/mfshome/$USER box:cluster-backup/daily  
\  
--dry-run --fast-list --checksum
```

3 Configure the Box remote with offline authorization

On the cluster, run `rclone` using its full path:

```
/groups/sterling/software-tools/rclone/rclone-v1.69.1-linux-  
amd64/rclone config
```

Press Enter to accept each default (shown in `<angle brackets>`):

No remotes found, make a new one?
n/s/q> n

name> box
Storage> box
client_id> <leave blank>
client_secret> <leave blank>
box_config_file> <leave blank>
access_token> <leave blank>

box_sub_type>
1 / user
2 / enterprise
box_sub_type> 2

Edit advanced config?
y/n> n

Use web browser to automatically authenticate?
y/n> n

1. **Open a new terminal** on the cluster (**keep the original rclone config terminal open**) and run that exact command. It will display a URL.
2. **Copy the URL** and **open it in your web browser** to complete the OAuth flow.
3. **Back in the new terminal**, it will print a **long token string**.
4. **Copy that token** and **return to the original terminal** to paste it at the rclone config prompt:

```
config_token> xxxxxxxxxxxxxxxxx...xxx
```

5. When asked, keep the remote:

```
Keep this "box" remote?  
y) y
```

6. Verify:

```
/groups/sterling/software-tools/rclone/rclone-v1.69.1-  
linux-amd64/rclone ls box:
```

4 Create the Box folder hierarchy

Run once on the cluster using the full rclone path:

```
# Parent backup folder  
/groups/sterling/software-tools/rclone/rclone-v1.69.1-linux-  
amd64/rclone mkdir box:cluster-backup  
  
# Subfolders  
/groups/sterling/software-tools/rclone/rclone-v1.69.1-linux-  
amd64/rclone mkdir box:cluster-backup/daily  
/groups/sterling/software-tools/rclone/rclone-v1.69.1-linux-  
amd64/rclone mkdir box:cluster-backup/archive  
/groups/sterling/software-tools/rclone/rclone-v1.69.1-linux-  
amd64/rclone mkdir box:cluster-backup/logs
```

Verify:

```
/groups/sterling/software-tools/rclone/rclone-v1.69.1-linux-  
amd64/rclone ls box:cluster-backup
```

5 Prepare the local environment

On the cluster, create a directory for logs:

```
mkdir -p ~/logs
```

6 Reference scripts

Sterling group members *do not* need to modify these; they live in `/groups/sterling/hpc-setup`.

6.1 A) backup.sh

```
#!/usr/bin/env bash

#
# -----

# Script: backup.sh
# Description:
#   Backs up local data to remote storage via rclone.
#   - Daily mirror of DATA_DIR to REMOTE_ROOT/daily. This is a
#     MIRROR, not an
#     incremental archive: local deletions propagate. History
#     comes from
#     --backup-dir, which moves replaced/deleted files to
#     REMOTE_ROOT/versions/<date>.
#   - Weekly snapshots (Sundays) to REMOTE_ROOT/archive/<date
#     >, copied server-side
#     from REMOTE_ROOT/daily so no data is re-uploaded from
#     the cluster.
#   - Prunes local logs older than 30 days (find -mtime +30,
#     so ~31 in practice)
#   - Keeps the newest REMOTE_KEEP_SNAPSHOTS weekly snapshots
#     (4 = about a month)
#   - Uploads logs to remote (copy, so remote logs are not
#     mirror-pruned)
# Usage:
#   backup.sh (override settings via environment variables as
#     needed)
#
# Exit status is non-zero on any failure, and a message is
#   written to stderr so
# cron mails it. Do not add --log-file to that path or the
# mail goes silent.
#
# Configuration (env overrides):
#   DATA_DIR           Local directory to back up (default: /
#     groups/.../$USER)
#   REMOTE_ROOT         Remote root for backups (default: box:
#     cluster-backup)
```

```

# RCLONE_BIN          Path to rclone binary (default: rclone-
#                      v1.69.1)
# LOG_DIR             Directory for local logs (default:
#                      $HOME/logs)
# SETTLE_MAX_WAIT     Seconds to wait for the remote listing
#                      to match the source
#                      before taking the weekly snapshot (
#                      default: 300)
#
# Author: Markus G. S. Weiss
# Date: 2025-05-05
#
-----

set -euo pipefail

# --- Configuration (override via env if desired)
# -----
# cron sets LOGNAME and HOME but is not required to set USER (
# see man 5 crontab),
# and 'set -u' would abort here before any log file exists.
# Derive it if absent.
: "${USER:=$(id -un)}"

: "${DATA_DIR:=/groups/sterling/mfshome/$USER}"
: "${REMOTE_ROOT:=box:cluster-backup}"
: "${RCLONE_BIN:=/groups/sterling/software-tools/rclone/rclone
-v1.69.1-linux-amd64/rclone}"
: "${LOG_DIR:=$HOME/logs}"
: "${SETTLE_MAX_WAIT:=300}"

DATE_STR=$(date +%F)
# Weekday captured once, next to DATE_STR, so a run that
# crosses midnight cannot
# disagree with itself about which day it is.
DOW=$(date +%u)
LOCK_FILE="$HOME/.backup_${USER}.lock"

# Common rclone options
RCLONE_OPTS="--fast-list --checksum --log-level WARNING"

```

```

# Retry settings
MAX_RETRIES=3
RETRY_DELAY=10

# How many weekly snapshots to keep. Counted, not dated: a run
# that is skipped
# (the lock is held, or a failure) would silently cost us a
# snapshot under a
# date cutoff, whereas a count keeps the newest four whenever
# they were taken.
REMOTE_KEEP_SNAPSHOTS=4

# Refuse a sync that would delete more than this many remote
# files in one run.
# Guards against mirroring an empty source (e.g. an unmounted
# $DATA_DIR) onto the
# remote. Raise it deliberately for a genuine bulk cleanup.
MAX_DELETE=100

# --- Setup
-----

# Ensure log directory exists
mkdir -p "$LOG_DIR"

# Prevent overlapping runs
exec 200>"$LOCK_FILE"
flock -n 200 || {
    echo "[$(date '+%F %T')] Another backup is already running.
    Exiting. Holder: $(cat "$LOCK_FILE" 2>/dev/null || echo
    unknown)" >> "$LOG_DIR/backup-$(date '+%F %T').log"
    echo "backup.sh: skipped on $(hostname) at $(date '+%F %T')
    -- another run holds $LOCK_FILE" >&2
    exit 1
}

# Record who holds the lock, so a stuck run can be identified
# rather than guessed at.
echo "pid=$$ host=$(hostname) started=$(date '+%F %T')" >&200

# Report any non-zero exit on stderr, which is the one channel
# cron will mail.

```

```

# rclone's --log-file sends everything else to the log,
# leaving stderr empty, so
# without this a persistent failure is completely silent.
on_exit() {
    local rc=$?
    if (( rc != 0 )); then
        echo "backup.sh FAILED (rc=$rc) on $(hostname) at $(date
            '+%F %T') -- see $LOG_DIR/backup-$DATE_STR.log" >&2
    fi
    exit "$rc"
}
trap on_exit EXIT

# --- Utility: retry wrapper
# -----
# rclone exit codes that more attempts cannot fix (rclone
# documents these as
# non-retryable):  2 syntax/usage    3 directory not found    4
#                  file not found
#                  7 fatal error (includes "--max-delete
#                  threshold reached")
#                  8 transfer limit reached
# Retrying 3 and 4 merely burns MAX_RETRIES x RETRY_DELAY
# seconds. Retrying 7 is
# actively DANGEROUS: rclone refuses a sync once --max-delete
# is hit, but each
# retry starts from the already-reduced state, so a blind
# retry loop simply grinds
# a mass deletion through in batches and reports success.
# Verified: with 150
# deletions pending and --max-delete 100, attempt 1 deletes
# 100 and exits 7, and
# attempt 2 finds only 50 left and succeeds.
NO_RETRY_CODES=" 2 3 4 7 8 "

retry() {
    local n=1 cmd="$*" rc logf="$LOG_DIR/backup-$DATE_STR.log"
    while true; do
        rc=0
        eval "$cmd" || rc=$?
        (( rc == 0 )) && return 0
    done
}

```

```

if [[ "$NO_RETRY_CODES" == "*" $rc "*" ]]; then
    echo "[$(date '+%F %T')] ERROR: Command failed with non-
        retryable exit $rc: $cmd" >> "$logf"
    return "$rc"
fi
if (( n >= MAX_RETRIES )); then
    echo "[$(date '+%F %T')] ERROR: Command failed after
        $MAX_RETRIES attempts (exit $rc): $cmd" >> "$logf"
    return "$rc"
fi
echo "[$(date '+%F %T')] WARN: Command failed (attempt $n/
    $MAX_RETRIES, exit $rc). Retrying in $RETRY_DELAY s..."
    >> "$logf"
    sleep "$RETRY_DELAY"
    ((n++))
done
}

# --- 1) Prune local logs older than N days
-----
prune_local_logs() {
    local retention_days=30 logf="$LOG_DIR/backup-$DATE_STR.log"
    echo "[$(date '+%F %T')] Pruning local logs older than
        $retention_days days..." >> "$logf"
    find "$LOG_DIR" -type f -name '*.log' -mtime +
        $retention_days -delete
    echo "[$(date '+%F %T')] Pruning local logs completed." >> "
        $logf"
}

# --- 2) Prune old remote snapshots
-----
# Keeps the newest N dated folders. Do NOT go back to 'rclone
    delete --min-age':
# that filters on each object's modification time, which
    rclone copy inherits from
# the source file, so it deleted long-untouched files from a
    snapshot taken the day
# before while leaving the empty dated folder behind to look
    healthy.

```

```

prune_remote_snapshots() {
    local logf="$LOG_DIR/backup-$DATE_STR.log" snap
    # RCLONE_OPTS is a flag list; split it so this direct (non-
    # eval) call can pass
    # it properly quoted.
    local -a opts snaps=()
    read -r -a opts <<< "$RCLONE_OPTS"

    # lsf --dirs-only yields "2026-08-23/" one per line.
    while read -r snap; do
        snap="${snap%/"
        # Ignore anything that is not a YYYY-MM-DD folder rather
        # than guessing at it.
        if [[ ! "$snap" =~ ^[0-9]{4}-[0-9]{2}-[0-9]{2}$ ]]; then
            echo "[$(date '+%F %T')] Skipping unrecognised archive
                entry: $snap" >> "$logf"
            continue
        fi
        snaps+=("$snap")
    done < <("$RCLONE_BIN" lsf --dirs-only "$REMOTE_ROOT/archive
        " "${opts[@]}" --log-file "$logf")

    if (( ${#snaps[@]} == 0 )); then
        echo "[$(date '+%F %T')] No snapshots to prune." >> "$logf"
        return 0
    fi

    # YYYY-MM-DD sorts lexicographically the same as
    # chronologically, so a reverse
    # sort puts the newest first and everything past the Nth is
    # surplus.
    mapfile -t snaps < <(printf '%s\n' "${snaps[@]}" | sort -r)
    echo "[$(date '+%F %T')] ${#snaps[@]} snapshot(s) present;
        keeping the newest $REMOTE_KEEP_SNAPSHOTS." >> "$logf"

    local i
    for (( i = REMOTE_KEEP_SNAPSHOTS; i < ${#snaps[@]}; i++ ));
    do
        echo "[$(date '+%F %T')] Purging snapshot ${snaps[i]}" >>
            "$logf"
    done
}

```

```

# purge, not delete: removes the directory too, so no
# empty shells accumulate.
retry "$RCLONE_BIN purge '$REMOTE_ROOT/archive/${snaps[i]}' $RCLONE_OPTS --log-file '$logf'"
done

echo "[$(date '+%F %T')] Pruned remote snapshots." >> "$logf"
"
}

# --- 3) Daily mirror + versioned deletions
-----
# 'sync' mirrors deletions, so --backup-dir is what gives this
# any history at all:
# replaced and deleted files are moved under versions/<date>
# instead of destroyed.
#
# The exclude is anchored with a leading slash. Without it,
# rclone matches the
# pattern at ANY depth, so every user directory named "envs"
# anywhere in the tree
# was silently omitted from the backup, not just the conda
# envs dir at the root.
backup_daily() {
    local src="$DATA_DIR" dest="$REMOTE_ROOT/daily" logf="
        $LOG_DIR/backup-$DATE_STR.log"

    # An existing-but-empty source is the dangerous case: rclone
    # only errors when the
    # directory is ABSENT. If the filesystem is not mounted (and
    # note setup.sh will
    # happily mkdir -p the mountpoint), sync would mirror the
    # emptiness and wipe the
    # remote in a run that reports success.
    if [[ ! -d "$src" ]] || [[ -z "$(ls -A "$src" 2>/dev/null)"
        ]]; then
        echo "[$(date '+%F %T')] ABORT: DATA_DIR '$src' is missing
            or empty -- refusing to sync." >> "$logf"
        return 1
    fi
}

```

```

echo "[$(date '+%F %T')] Starting daily backup from $src to
    $dest..." >> "$logf"
retry "$RCLONE_BIN sync '$src' '$dest' $RCLONE_OPTS \
    --exclude '/envs/**' --max-delete $MAX_DELETE \
    --backup-dir '$REMOTE_ROOT/versions/$DATE_STR' --log-file
    '$logf'"
echo "[$(date '+%F %T')] Daily backup completed." >> "$logf"
}

# --- Utility: let the remote listing catch up
# -----
# The weekly snapshot copies server-side from daily/, which
# rclone reads through a
# listing. If the object store has not yet published a just-
# finished upload, the
# snapshot would silently miss it. Wait until the remote
# reports what the source
# holds -- object count and byte total, so this is a listing
# on each side with no
# hashing -- backing off up to SETTLE_MAX_WAIT. Doubles as a
# check that the sync
# actually transferred everything it should have.
wait_for_daily_to_settle() {
    local logf="$1" waited=0 delay=5 want got
    local -a opts; read -r -a opts <<< "$RCLONE_OPTS"

    want=$("$RCLONE_BIN" size "$DATA_DIR" --json --exclude '/
    envs/**' "${opts[@]}" 2>/dev/null)
    if [[ -z "$want" ]]; then
        echo "[$(date '+%F %T')] WARN: could not size $DATA_DIR;
            snapshotting without the settle check." >> "$logf"
        return 1
    fi

    while ;; do
        got=$("$RCLONE_BIN" size "$REMOTE_ROOT/daily" --json "${
            opts[@]}" 2>/dev/null)
        if [[ "$want" == "$got" ]]; then
            echo "[$(date '+%F %T')] Remote daily matches source
                after ${waited}s: $want" >> "$logf"
            return 0
        fi
    done
}

```

```

fi
if (( waited >= SETTLE_MAX_WAIT )); then
{
    echo "[$(date '+%F %T')] WARN: remote daily still
        differs after ${waited}s."
    echo "[$(date '+%F %T')]          source=$want"
    echo "[$(date '+%F %T')]          remote=$got"
    echo "[$(date '+%F %T')]          Taking the snapshot
        anyway; it mirrors whatever daily holds."
} >> "$logf"
return 1
fi
# never sleep past the bound, so SETTLE_MAX_WAIT is a real
# ceiling
(( delay > SETTLE_MAX_WAIT - waited )) && delay=$((
    SETTLE_MAX_WAIT - waited ))
echo "[$(date '+%F %T')] Remote daily not settled yet;
    retrying in ${delay}s (waited ${waited}s)." >> "$logf"
sleep "$delay"
waited=$(( waited + delay ))
(( delay < 60 )) && delay=$(( delay * 2 ))
done
}

# --- 4) Weekly snapshot (Sundays)
# -----
# Snapshots copy SERVER-SIDE from the daily mirror rather than
# re-uploading the
# whole home directory from the cluster every week. rclone's
# Box backend supports
# server-side copy, so this moves no bytes over the uplink and
# does no local
# hashing. The old local->Box copy made the Sunday run far
# longer than any other;
# once it passed 24h the flock below silently cancelled Monday
# 's backup entirely
# (and every following day until it finished), with no email
# and one line in a log.
#
# It also means the snapshot reflects exactly what was backed
# up, and the

```

```

# --exclude is no longer needed here because daily/ is already
  filtered.
snapshot_weekly() {
  local logf="$LOG_DIR/snapshot-$DATE_STR.log"
  if [[ "$DOW" != "7" ]]; then
    echo "[$(date '+%F %T')] Not Sunday (weekday $DOW) -- no
      weekly snapshot." >> "$LOG_DIR/backup-$DATE_STR.log"
    return 0
  fi
  local src="$REMOTE_ROOT/daily" dest="$REMOTE_ROOT/archive/
    $DATE_STR"
  # A mismatch is reported, not fatal: the snapshot mirrors
    daily either way.
  wait_for_daily_to_settle "$logf" || true
  echo "[$(date '+%F %T')] Starting weekly snapshot from $src
    to $dest..." >> "$logf"
  retry "$RCLONE_BIN copy '$src' '$dest' $RCLONE_OPTS --log-
    file '$logf'"
  echo "[$(date '+%F %T')] Weekly snapshot completed." >> "
    $logf"
}

# --- 5) Upload logs
-----
# 'copy', not 'sync': sync mirrored the local 31-day prune
  onto the remote, so the
# remote log archive could never outlive the local one, and it
  also deleted any
# remote-only object (e.g. compressed rotations) that had no
  local counterpart.
#
# The completion lines are written BEFORE the transfer so the
  uploaded copy of
# today's log actually contains them, and today's log is
  excluded from its own
# upload so rclone is not reading a file it is concurrently
  appending to.
upload_logs() {
  local src="$LOG_DIR" dest="$REMOTE_ROOT/logs" logf="$LOG_DIR
    /backup-$DATE_STR.log"

```

```

echo "[$(date '+%F %T')] Uploading logs from $src to $dest
..." >> "$logf"
echo "[$(date '+%F %T')] Script finished successfully." >> "
$logf"
retry "$RCLONE_BIN copy '$src' '$dest' $RCLONE_OPTS --log-
file '$logf'"
}

# --- Main
-----

# The backup runs FIRST. Pruning is maintenance; under 'set -e
# ' a failed prune used
# to abort the run before backup_daily ever started, and
# because the archive folder
# is only created by snapshot_weekly, a deployment where it
# did not yet exist died
# at the prune every single night and never backed anything up
.
main() {
    backup_daily
    snapshot_weekly
    prune_local_logs
    # Pruning must never cost us the backup, so its failure is
    # reported and swallowed.
    prune_remote_snapshots || \
        echo "[$(date '+%F %T')] WARN: remote snapshot prune
        failed; continuing." >> "$LOG_DIR/backup-$DATE_STR.log"
    upload_logs
}

main "$@"

# ---Log Rotation (optional)
-----

# For home-directory logs, add ~/.config/logrotate/backup:
# $HOME/logs/*.log {
#     daily
#     rotate 30
#     compress
#     missingok

```

```
#   notifempty
#   copytruncate
# }
```

Make it executable:

```
chmod +x /groups/sterling/hpc-setup/scripts/backup.sh
```

6.2 B) crontab-backup

```
#
# -----
# Crontab: sterling's backup jobs
# Description:
#   Runs the master backup.sh every day, with all pruning and
#   log-uploads
#   handled internally in that script.
#
# Author: Markus G. S. Weiss
# Date:   2025-05-05
#
# -----

SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
#MAILTO=$USER@utdallas.edu

# Run backup.sh daily at 02:00
0 2 * * * /groups/sterling/hpc-setup/scripts/backup.sh
# End of crontab
#
#####
```

7 Install the cron job

On the cluster, install the pre-written cron script.

Note: `crontab <file>` *replaces* your entire crontab. It does not merge, and it does not warn. Both commands below append instead, so any jobs you already have survive.

For the central deployment:

```
(crontab -l 2>/dev/null; cat /groups/sterling/hpc-setup/
scripts/crontab-backup) | crontab -
```

From a git clone, rewrite the hardcoded path first:

```
(crontab -l 2>/dev/null; sed "s|/groups/sterling/hpc-setup|$(
pwd)|" scripts/crontab-backup) | crontab -
```

Verify – check the result before trusting it:

```
crontab -l
```

8 Monitoring & Maintenance

- View logs (live tail):

```
tail -f ~/logs/backup-$(date +%F).log
```

- Clean up local logs older than 30 days:

```
find ~/logs -type f -mtime +30 -delete
```

- Test restores:

```
/groups/sterling/software-tools/rclone/rclone-v1.69.1-
linux-amd64/rclone copy \
box:cluster-backup/daily/path/to/file /tmp && \
diff /tmp/file /groups/sterling/mfshome/$USER/path/to/
file
```

- **Error notifications:** Cron will email stderr/stdout to `$USER@yourdomain.com`. For advanced alerting, grep logs for `ERROR` and pipe to mail or integrate with Slack.

9 Additional Notes

- **Security & permissions:** Do *not* check `~/.config/rclone/rclone.conf` into any shared repositories—it contains tokens.
- **Data encryption:** Consider using an rclone `crypt` wrapper for encryption at rest.
- **API rate limits:** Box enforces API quotas. Tweak `-transfers`, `-checkers`, or add `-tpslimit 3` if you hit rate-limit errors.
- **Network/firewall:** Ensure outbound HTTPS (port 443) is open. If behind a proxy, set `HTTPS_PROXY` or use `-proxy`.
- **Monthly or quarterly snapshots:** Extend the weekly logic with checks like:

```
if [[ "$(date +%d)" == "01" ]]; then
    ... # monthly archive
fi
```

- **Upstream docs:** Official rclone Box backend documentation: <https://rclone.org/box/>

10 Summary

In this tutorial, you have:

- **Configured** the Box remote on a headless cluster node via offline authorization.
- **Created** a clear Box folder hierarchy (`cluster-backup/{daily,archive,logs}`) for organized storage.
- **Prepared** a local log directory and referenced centrally maintained backup and cron scripts.
- **Written** a robust `backup.sh` that mirrors daily (deleted and replaced files are kept under `versions/`), snapshots weekly, and uploads logs.
- **Scheduled** the backup using a `crontab`, including log rotation and snapshot cleanup.
- **Implemented** monitoring, restore procedures, and maintenance routines (log pruning, error alerts).

- **Added** best-practice notes on dry-runs, version checks, security, API-rate limits, and firewall considerations.

Great work! Your Sterling group directory (`‘/groups/sterling/mfshome/USER’`) *is now backed up to Bo*.
**not *include your login home‘/home/USER’* — only the mfshome tree is copied.